

A AURORA AJUSTA A TRAJETÓRIA NA APROXIMAÇÃO A MARTE

ALVARO
MARTINS
CRISTIANE
FLUXO DE DA
DA
APROXIMAÇÃO

LISTA DE FIGURAS

Figura 1 - Página de versões do MinGW-w64	17
Figura 2 - Instalador baixado (esquerda) e tela de seleção (direita)	18
Figura 3 - Instalação dos itens selecionados.....	18
Figura 4 - Página da plataforma VSCode.....	19
Figura 5 – Termo de instalação do VSCode	19
Figura 6 – As tarefas do VSCode.....	20
Figura 7 – Preparação para instalar o VSCode.....	20
Figura 8 – Conclusão da instalação do VSCode	21
Figura 9 – Tela principal do VSCode.....	22
Figura 10 – Seleção de “Abrir Pasta” no VSCode	23
Figura 11 – Seleção das pastas do projeto no VSCode	24
Figura 12 – Criando o primeiro programa no VSCode	25
Figura 13 – Criando o primeiro arquivo na pasta do projeto	26
Figura 14 – Tela final do código	27
Figura 15 – Ícone para verificação do código.....	27
Figura 16 – Código compilado e executado com sucesso	28

LISTA DE TABELAS

Tabela 1 – Tipo de dado x tamanho em bit	12
Tabela 2 – Categorias de operadores	14
Tabela 3 – Categorias de operadores (nível)	15

EMANSP

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 – Forma geral de um programa em C	9
Código-fonte 2 – Exemplo de declaração de variáveis em um programa em C	10
Código-fonte 3 – Exemplo de mapeamento seguro	12
Código-fonte 4 – Exemplo de qualificadores de tipo	13
Código-fonte 5 – Exemplo de ordem de avaliação	16
Código-fonte 6 – Passos para instalação no Linux	21
Código-fonte 7 – Passos para instalação no macOS	22
Código-fonte 8 – Hello World! Código em C	26
Código-fonte 9 – Exemplo de while no código em C	29
Código-fonte 10 – Exemplo de do...while no código em C	29
Código-fonte 11 – Exemplo de (loop) for no código em C	30
Código-fonte 12 – Exemplo de break e continue	30
Código-fonte 13 – Exemplo de if, else if e else no código em C	31
Código-fonte 14 – Exemplo de switch case no código em C	32
Código-fonte 15 – Exemplo de protótipo, definição e chamada no código em C	33
Código-fonte 16 – Exemplo de função com retorno no código em C	33
Código-fonte 17 – Exemplo de função somente com retorno no código em C	34
Código-fonte 18 – Exemplo de função somente com parâmetros no código em C	35
Código-fonte 19 – Exemplo de função sem parâmetros e sem retorno no código em C	35
Código-fonte 20 – Exemplo de vetor no código em C	36
Código-fonte 21 – Exemplo de pilha (stack) no código em C	37
Código-fonte 22 – Exemplo <i>buffer</i> “circular” da estrutura fila (queue) no código em C	37
Código-fonte 23 – Exemplo de lista encadeada (linked list) no código em C	38

SUMÁRIO

1 OVERVIEW DA LINGUAGEM C	7
1.1 Introdução	7
1.2 História da linguagem C	7
1.3 Por que C é chamada de “linguagem de médio nível”?	8
1.4 C e a programação estruturada.....	8
1.5 A forma geral de um programa em C	8
1.6 Palavras-chave e forma canônica em C.....	9
2 PROGRAMANDO EM C: VARIÁVEIS, TIPOS, MODIFICADORES, OPERADORES E EXPRESSÕES	9
2.1 Variáveis, escopo, duração e ligação	10
2.2 Tipos de dados.....	11
2.2.1 Tipos fundamentais	11
2.2.2 Tamanhos típicos por plataforma/ABI	11
2.2.3 Inteiros de largura fixa (<stdint.H>).....	12
2.2.4 Booleano	12
2.3 “Modificadores” de declaração: <i>storage class</i> e qualificadores de tipo	13
2.4 Operadores e expressões	13
2.4.1 Classes de operadores.....	14
2.4.2 Precedência e associatividade	14
2.4.3 Sequenciamento (ordem de avaliação)	15
3 CONFIGURANDO O AMBIENTE - VSCODE.....	16
3.1 O que vamos instalar	16
3.2 Instalação por sistema – Windows (recomendado).....	17
3.3 Instalação por sistema – Linux (Ubuntu/ Debian).....	21
3.4 Instalação por sistema – macOS (clang/LLDB).....	21
3.5 Configurações e extensões no VSCode.....	22
4 PROGRAMANDO EM C: MEU PRIMEIRO PROGRAMA – HELLO WORLD!	24
5 PROGRAMANDO EM C: LAÇOS DE REPETIÇÃO E CONDICIONAIS	28
5.1 Laços de repetição (<i>loop</i>): <i>while</i> , <i>do...while</i> e <i>for</i>	28
5.1.1 <i>while</i> — 0 ou mais repetições	29
5.1.2 <i>do...while</i> — 1 ou mais repetições	29
5.1.3 <i>for</i> — controle compacto.....	29
5.1.4 <i>break</i> e <i>continue</i> (usar com critério)	30
5.2 Condicionais: <i>true</i> ou <i>false</i> , <i>if</i> , <i>else if</i> e <i>else</i>	30
5.2.1 Verdadeiro e falso em C.....	31
5.2.2 <i>if</i> , <i>else if</i> , <i>else</i>	31
5.2.3 <i>switch case</i> (seleção múltipla).....	31
5.3 Funções: Protótipo, definição, chamada e tipos.....	32
5.3.1 Protótipo, definição e chamada	32
5.3.2 Função com retorno e com parâmetros.....	33
5.3.3 Função com retorno e sem parâmetros.....	34
5.3.4 Função sem retorno (<i>void</i>) e com parâmetros	34
5.3.5 Função sem retorno (<i>void</i>) e sem parâmetros	35

6 PROGRAMANDO EM C: ESTRUTURAS LINEARES – LISTAS, FILAS E PILHAS	36
6.1 O que é um vetor?	36
6.2 Pilha (<i>stack</i>)	36
6.3 Fila (<i>Queue</i>)	37
6.4 Lista encadeada (<i>linked list</i>)	37
CONCLUSÃO	38
REFERÊNCIAS	40

EMSE

1 OVERVIEW DA LINGUAGEM C

1.1 Introdução

Neste capítulo, vamos embarcar em uma jornada pelos fundamentos da linguagem C, a base sólida sobre a qual grande parte da computação moderna foi construída. Antes de explorar códigos mais complexos, é essencial compreender como a informação pode ser organizada, manipulada e percorrida de forma eficiente. Aqui surgem as estruturas lineares, modelos sequenciais que permitem representar listas de tarefas, filas de atendimento, pilhas de operações e tantas outras situações presentes no dia a dia da computação (e da vida real).

Agora apresentaremos uma visão introdutória da linguagem C: suas origens, usos e filosofia de projeto, servindo como base para quem está iniciando.

1.2 História da linguagem C

A linguagem C foi criada no início da década de 1970 por Dennis M. Ritchie nos *Bell Labs*, em estreita relação com o sistema operacional UNIX. Ela evoluiu a partir de BCPL e B, ganhando tipagem e recursos que a tornaram adequada para implementar partes do próprio UNIX, o que consolidou seu uso como linguagem de sistemas.

Durante os anos 1970 e 1980, o livro *The C Programming Language* (o “K&R”) serviu amplamente como referência. Com a multiplicação de compiladores, surgiu a necessidade de um padrão formal, culminando no ANSI C (1989) e na norma internacional ISO/IEC 9899:1990 (C90). A padronização prosseguiu (C99, C11, C17 e C23), mantendo a essência da linguagem e documentando extensões e correções.

O contexto técnico do UNIX, portabilidade, simplicidade e ferramentas pequenas que se combinam, explica muito da filosofia de C e de seu sucesso prático. O artigo clássico de Ritchie e Thompson (1974) descreve a arquitetura do UNIX e ajuda a entender por que C foi uma escolha natural para escrever o sistema e suas bibliotecas.

Historicamente, essa linguagem foi muito empregada para programas de sistemas: operacionais, utilitários, interpretadores, editores, compiladores e gerenciadores de banco de dados. Com a evolução da norma e dos compiladores, difundiu-se também em aplicações gerais, bibliotecas de desempenho e software

embarcado. A portabilidade da norma ISO/IEC 9899 e o ecossistema de compiladores disponíveis para diferentes plataformas explicam essa difusão.

1.3 Por que C é chamada de “linguagem de médio nível”?

C costuma ser descrita como uma linguagem de “médio nível” porque combina abstrações de alto nível (funções, estruturas de controle e tipos compostos) com acesso de baixo nível (endereço de memória, manipulação de bits e interação direta com o *hardware*). Essa combinação permite desenvolver tanto o *software* de sistemas (núcleos de SO, drivers e bibliotecas) quanto aplicações portáteis.

A portabilidade foi reforçada pela padronização ANSI/ISO. A norma ISO/IEC 9899 (e seus *drafts* públicos, como o N1570 do C11) definem rigorosamente a linguagem e a biblioteca padrão, favorecendo a compatibilidade entre implementações.

1.4 C e a programação estruturada

C incentiva um estilo estruturado de programação: sequência, seleção (*if/else* e *switch*) e iteração (*for*, *while* e *do-while*), com organização em blocos `{}`. Ainda que não ofereça funções aninhadas, a modularização via arquivos “.h” e “.c” e a clara separação entre interface e implementação favorecem a legibilidade e a manutenção.

Os blocos `{...}` são qualquer sequência de comandos entre chaves que formam uma unidade lógica. Por exemplo, o corpo de um `if (x < 10) {...}` é um bloco único, executado apenas quando a condição é verdadeira. Essa unidade ajuda a pensar “o que esta rotina realmente faz?”, ao invés de espalhar saltos e rótulos (Kernighan, 1988).

1.5 A forma geral de um programa em C

Um programa em C é composto por declarações globais (opcionais) e um conjunto de funções, dentre as quais `main` é o ponto de entrada. Cada função tem um tipo de retorno, um nome, uma lista de parâmetros e um bloco delimitado por chaves `{...}` com os comandos que serão executados. Em projetos reais, é comum dividir a

interface em arquivos de cabeçalho (.h) e a implementação em arquivos-fonte (.c), favorecendo modularidade e compilação incremental.

```
/* declarações globais e #includes */

tipo main(lista_de_parâmetros)
{
    /* sequência de comandos */
}

tipo fl(lista_de_parâmetros)
{
    /* sequência de comandos */
}

/* ... outras funções ... */
```

Código-fonte 1 – Forma geral de um programa em C
Fonte: Elaborado pelo autor (2025)

1.6 Palavras-chave e forma canônica em C

A especificação de C define um conjunto fechado de palavras-chave (como `int`, `double`, `if`, `while`, `struct`, `typedef`, `const`, `signed`, `void`, `volatile`, entre outras). Essas palavras são reservadas, ou seja, não podem ser usadas como nomes de variáveis ou funções, e aparecem em minúsculas por convenção. Além das palavras-chave, algumas implementações oferecem extensões específicas (por exemplo, para detalhes de *hardware*), mas o comportamento portátil é o que está descrito na norma.

Todo programa em C possui uma ou mais funções; a única obrigatória é `main`, que atua como ponto de entrada. Em geral, `main` coordena o fluxo do programa chamando outras funções, cada uma responsável por determinada tarefa bem definida.

2 PROGRAMANDO EM C: VARIÁVEIS, TIPOS, MODIFICADORES, OPERADORES E EXPRESSÕES

Veremos aqui como a linguagem C modela dados e computações: começaremos por variáveis (escopo, duração e ligação) e pelos tipos fundamentais e

derivados, destacando a influência do modelo de dados da plataforma nos tamanhos (ex.: long, ponteiros e long double). Em seguida, estudaremos os especificadores de classe de armazenamento (static, extern etc.) e os qualificadores de tipo (const, volatile, restrict, _Atomic etc.), que determinam visibilidade, vida útil e regras de acesso. Por fim, cobriremos operadores e expressões, com ênfase em precedência/associatividade e no sequenciamento definido pelo padrão, para escrever o código correto, legível e portátil.

2.1 Variáveis, escopo, duração e ligação

Uma variável é um objeto com tipo e identificador que ocupa memória e armazena um valor. Em C, o comportamento de uma variável depende de três eixos: escopo (onde o nome é visível: bloco, arquivo ou parâmetro), duração de armazenamento (quanto tempo o objeto “vive”: automático, estático, por *thread* ou dinâmico) e ligação (se o nome é visível em outras unidades de tradução: nenhuma, interna ou externa). Esses conceitos são definidos formalmente na norma, em especial nas seções de tipos/declarações e no capítulo de expressões.

Exemplo:

```
/*          arquivo          .c          */
int g = 42;          // duração estática, ligação externa
static int si = 7; // duração estática, ligação interna
(só          neste          .c)

void          f(void)          {
int x = 0;          // duração automática (vida = execução
def)
static int c = 0; // duração estática (preserva valor entre
chamadas)
c++;          x++;
}
```

Código-fonte 2 – Exemplo de declaração de variáveis em um programa em C
Fonte: Elaborado pelo autor (2025)

A duração/ligação acima é controlada pelos *storage-class specifiers*, como static e extern.

2.2 Tipos de dados

Em C, o tipo determina como um valor é representado na memória e quais operações são válidas sobre ele. A linguagem oferece tipos fundamentais, inteiros com e sem sinal (`char`, `short`, `int`, `long` e `long long`), ponto flutuante (`float`, `double` e `long double`), `enum` e `void`, e permite compor tipos derivados, como ponteiros, arrays, `struct`, `union` e tipos de função. Os tamanhos e faixas dos tipos não são fixos pelo padrão; são *implementation-defined* e, por isso, consultamos os cabeçalhos `<limits.h>` e `<float.h>` para limites portáteis, além de `<stdint.h>` quando precisamos de larguras exatas (por exemplo, `int32_t` ou `uint64_t`).

Também há suporte nativo a booleano via `_Bool` (e `bool` em `<stdbool.h>`). A seguir organizaremos esses elementos, ressaltando a relação entre modelo de dados da plataforma e portabilidade do código.

2.2.1 Tipos fundamentais

- Inteiros com e sem sinal: `char`, `short`, `int`, `long` e `long long` (cada um pode ser `signed/unsigned`). Os limites portáteis estão em `<limits.h>`;
- Ponto flutuante: `float`, `double` e `long double` (características/precisão em `<float.h>`);
- Enumeração: `enum` define constantes inteiras nomeadas;
- `void`: ausência de tipo/valor (ex.: função sem valor de retorno).

2.2.2 Tamanhos típicos por plataforma/ABI

O padrão C não fixa tamanhos absolutos (exceto nos inteiros de largura fixa de `<stdint.h>`). Por conta disso, os tamanhos dependem do modelo de dados da ABI (ILP32, LP64 e LLP64). É importante verificar sempre com `sizeof`.

A tabela a seguir apresenta a relação entre os sistemas operacionais e o tamanho de cada tipo de dado ocupado.

Tipo (C)	ILP32 (32-bit Linux/i386)	LP64 (Linux/macOS 64-bit, SysV AMD64)	LLP64 (Windows 64-bit / MSVC)
<code>char</code>	1	1	1

short	2	2	2
int	4	4	4
long	4	8	4
long long	8	8	8
ponteiro (void)	4	8	8
size_t	4	8	8
_Bool	1 (usual)	1 (usual)	1
float	4	4	4
double	8	8	8
long double	12 ou 16*	16 (80-bit com <i>padding</i>)	8 (igual a double)

Tabela 1 – Tipo de dado x tamanho em bit
Fonte: Elaborada pelo autor (2025)

2.2.3 Inteiros de largura fixa (<stdint.h>)

Quando disponíveis, os inteiros de largura fixa garantem largura exata: `int8_t/uint8_t` (8 bits), `int16_t/uint16_t` (16), `int32_t/uint32_t` (32) e `int64_t/uint64_t` (64). É importante usar as variantes `*_least*` e `*_fast*` quando precisar de “pelo menos N bits” ou da opção mais rápida.

2.2.4 Booleano

Em C, o tipo booleano é `_Bool`. Além disso, o cabeçalho `<stdbool.h>` (desde C99) fornece os *macros* `bool`, `true` e `false` (implementações e versões recentes podem ajustar a forma de disponibilização, mas o tipo subjacente é `_Bool`).

```
#include <stdint.h>
#include <stdbool.h>

typedef struct {
    int32_t id;
    float score;
    bool ativo; // mapeia para _Bool
} Registro;
```

Código-fonte 3 – Exemplo de mapeamento seguro
Fonte: Elaborado pelo autor (2025)

2.3 “Modificadores” de declaração: *storage class* e qualificadores de tipo

Em C não há “modificadores de acesso” no sentido OO (público/privado). O que existe são (ISO/IEC 9899, 2023):

- Especificadores de classe de armazenamento (*storage-class*): controlam a duração e a ligação do nome (auto, register, static, extern e _Thread_local).
- Qualificadores de tipo: impõem restrições semânticas sobre leitura/escrita/*aliasing* do objeto (const, volatile, restrict e _Atomic).

```
static int cache_hits; // global com ligação interna
(arquivos .c diferentes não veem)
void escala(size_t n, float * restrict a, const float k)
{
    for (size_t i = 0; i < n; ++i) a[i] *= k;
}
volatile unsigned *UART_STAT = (unsigned*)0x40001000u; //
I/O mapeado em memória
```

Código-fonte 4 – Exemplo de qualificadores de tipo
Fonte: Elaborado pelo autor (2025)

2.4 Operadores e expressões

Estudaremos como C combina valores por meio de operadores para formar expressões, que são a base de toda computação no programa. Veremos as classes de operadores (aritméticos, relacionais, lógicos, bit a bit, atribuição, condicionais e de acesso), a precedência e a associatividade que determinam a ordem de avaliação e, sobretudo, as regras normativas de sequenciamento introduzidas nas versões modernas do padrão (o modelo “*sequenced before/after*”), essenciais para evitar comportamento indefinido ao misturar efeitos colaterais em uma mesma expressão.

Ao final, você saberá interpretar e escrever expressões corretas e legíveis, escolhendo parênteses quando necessário para tornar a intenção inequívoca.

2.4.1 Classes de operadores

Em C, as expressões são construídas combinando categorias de operadores (aritméticos, relacionais/igualdade, lógicos, bit a bit, atribuição, condicional e utilitários, como `sizeof`, `cast`, acesso a membro e vírgula). Essa taxonomia e a sintaxe de cada operador estão consolidadas nas referências de linguagem e resumos técnicos, conforme pode ser visto na tabela a seguir (Cppreference, 2025).

Categoria	Operadores / Exemplos
Aritméticos	<code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> , <code>%</code>
Incremento / Decremento	<code>++</code> , <code>--</code> (prefixo/sufixo)
Relacionais / Igualdade	<code><</code> , <code><=</code> , <code>></code> , <code>>=</code> , <code>==</code> , <code>!=</code>
Lógicos	<code>&&</code> , <code>^</code>
Bit a bit	<code>&</code> , <code>^</code>
Atribuição	<code>=</code> , <code>+=</code> , <code>-=</code> , <code>*=</code> , <code>/=</code> , <code>%=</code> , <code><<=</code> , <code>>>=</code> , <code>&=</code> , <code>^=</code> , <code>`</code>
Condicional	<code>?:</code>
Outros	<code>sizeof</code> <code>(type) cast</code> <code>&</code> (endereço) <code>*</code> (indireção) <code>.</code> / <code>-></code> (membro) <code>,</code> (vírgula)

Tabela 2 – Categorias de operadores
Fonte: Elaborada pelo autor (2025)

2.4.2 Precedência e associatividade

A precedência determina a “força de ligação” entre operadores quando aparecem juntos, enquanto a associatividade define como operadores do mesmo nível são agrupados.

A hierarquia prática a seguir baseia-se na tabela clássica adotada na documentação técnica e está de acordo com a gramática normativa do padrão, conforme a cláusula 6.5 do C (Cppreference, 2025).

Nível (alto→baixo)	Operadores (exemplos)	Observações
1	(), [], ., ->, ++, -- (prefixo), +/- (unário), !, ~, (type), *, &, sizeof	Unários & acesso
2	*, /, %	Multiplicativos
3	+, -	Aditivos
4	<<, >>	Deslocamentos
5	<, <=, >, >=	Relacionais
6	==, !=	Igualdade
7	&	AND bit a bit
8	^	XOR bit a bit
9	`	`
10	&&	AND lógico
11	`	`
12	?:	Condiciona
13	=, +=, -=, *=, /=, %=, <<=, >>=, &=, ^=, `	=`
14	,	Vírgula

Tabela 3 – Categorias de operadores (nível)
Fonte: Elaborada pelo autor (2025)

2.4.3 Sequenciamento (ordem de avaliação)

Desde C11, a norma substitui “pontos de sequência” por um modelo formal de “*sequenced before/after*”, para ordenar efeitos e avaliações. Na prática, é importante evitar expressões que leem e escrevem a mesma variável sem um ponto claro de sequenciamento (ex.: `i = i++ + 1;`), conforme a cláusula 6.5 de C (*expressions* e guias técnicos).

```
int f1(void), f2(void);
int z = f1() + f2();    // a linguagem NÃO garante quem roda
                        primeiro (f1 ou f2)

int i = 1;
```

```
// i = i++ + 1;          // EXEMPLO A EVITAR: lê e escreve i sem
// sequenciamento → indefinido
i++; i = i + 1;          // faça em passos separados (definido)
```

Código-fonte 5 – Exemplo de ordem de avaliação
Fonte: Elaborado pelo autor (2025)

3 CONFIGURANDO O AMBIENTE - VSCODE

Aqui, vamos aprender a preparar um ambiente profissional de C no *Visual Studio Code*: instalar o VS Code e a extensão C/C++ (*IntelliSense*, navegação e depuração), escolher e instalar o *toolchain* correto a cada sistema, MSYS2/MinGW-w64 (UCRT64) no Windows, GCC/GDB no Linux e Clang/LLDB no macOS, e validar tudo no terminal integrado. Em seguida, configuraremos *build* e *debug* com *tasks.json/launch.json*, além do fluxo com *CMake Tools* para projetos maiores, finalizando com um “Hello, C” depurado passo a passo e um checklist de solução de problemas (PATH, detecção do compilador e executável de saída).

Ao terminar, você conseguirá compilar, executar e depurar programas em C de forma portátil nas três plataformas principais, com base em guias oficiais.

3.1 O que vamos instalar

1. VS Code + extensão C/C++ (Microsoft) para *IntelliSense*, depuração (GDB/LLDB) e navegação.
2. Compilador e depurador adequados ao seu SO:
 - a. **Windows** (recomendado): MSYS2/MinGW-w64 (UCRT64) → gcc/g++ e gdb (<http://sourceforge.net/projects/mingw/>).
 - b. **Windows** (alternativa): MSVC (cl.exe) com tutorial oficial do VS Code (<https://code.visualstudio.com/docs/cpp/config-msvc>).
 - c. **Linux (Ubuntu/Debian)**: build-essential (gcc, make, gdb) (<https://linuxize.com/post/how-to-install-gcc-on-ubuntu-20-04/>).
 - d. **macOS**: Xcode Command Line Tools (clang/lldb).
3. (Opcional) *CMake Tools* para projetos maiores e multi-arquivo.

3.2 Instalação por sistema – Windows (recomendado)

Vamos apresentar agora os passos para a instalação do compilador e depurador para iniciar a programação em C.

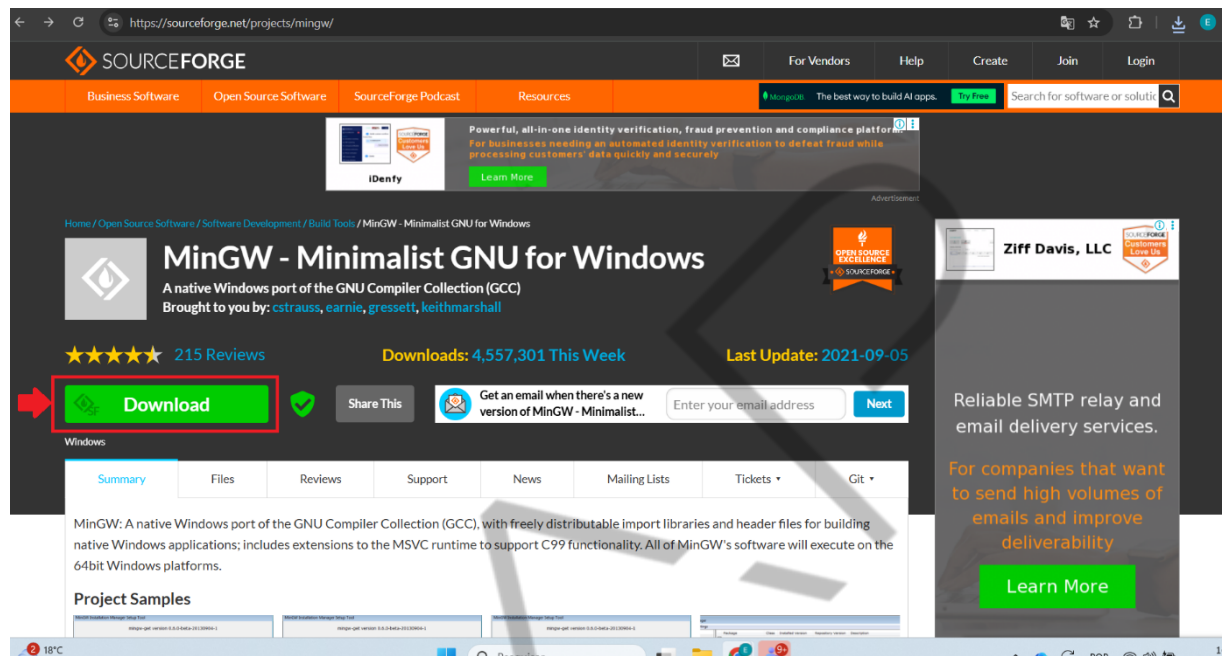


Figura 1 - Página de versões do MinGW-w64
Fonte: Elaborada pelo autor (2025)

Devemos selecionar e baixar a última versão do instalador, para evitar quaisquer problemas de versão ou compatibilidade.

Assim que finalizar o download, vá até a pasta que o arquivo foi baixado e inicie a instalação, seguindo os seguintes passos:

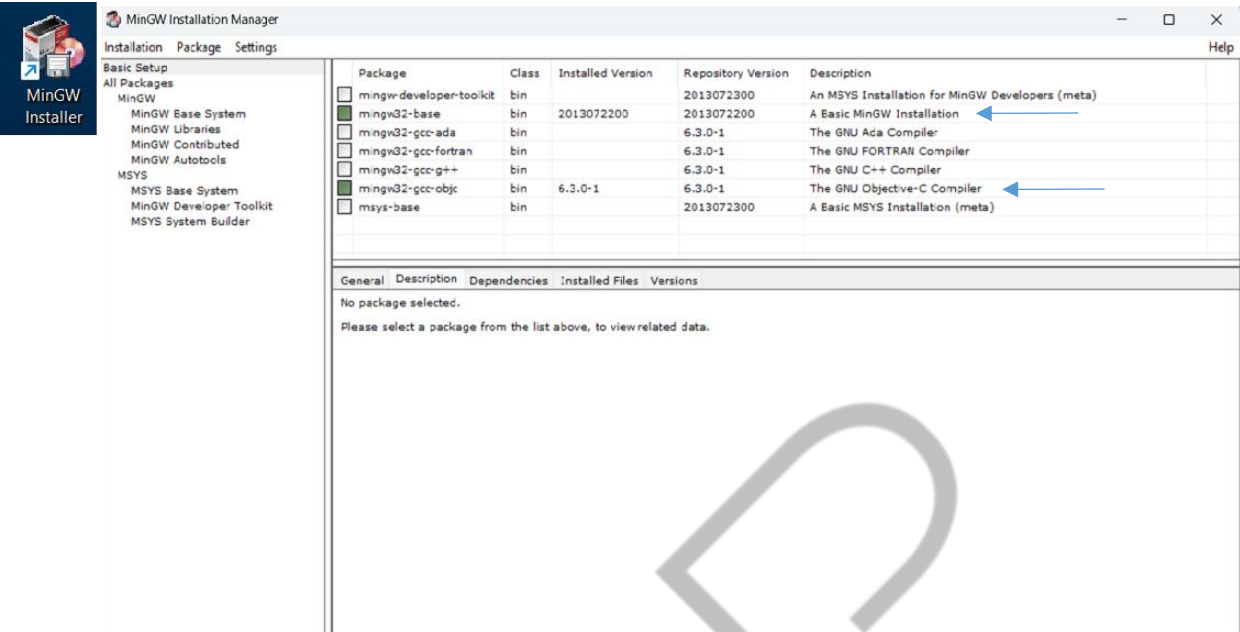


Figura 2 - Instalador baixado (esquerda) e tela de seleção (direita)
Fonte: Elaborada pelo autor (2025)

Na figura anterior, pode-se verificar que os seguintes itens foram selecionados no *basic setup*:

- Mingw32-base;
- Mingw32-gcc-objc.

Após a seleção dos itens, deve-se clicar no “Installation” e depois em “Apply Changes” e aguardar a instalação finalizar ou clicar em *Close* ao final.

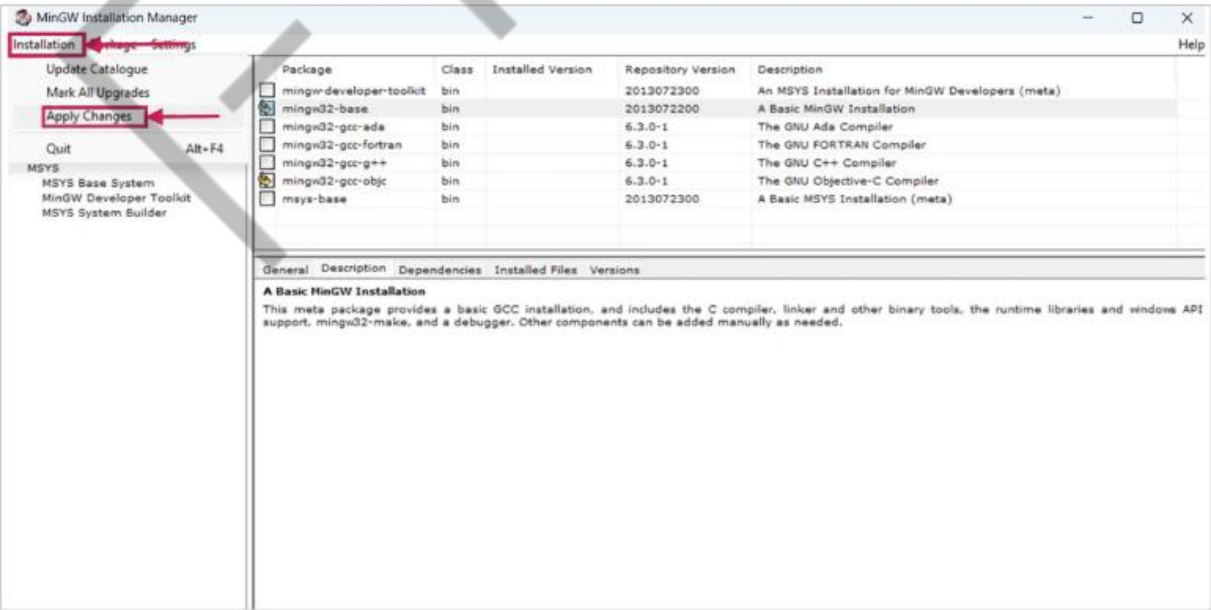


Figura 3 - Instalação dos itens selecionados
Fonte: Elaborada pelo autor (2025)

Em seguida, iniciaremos as instalações da plataforma VSCode. A figura abaixo apresenta a página inicial e o botão de instalação da plataforma.

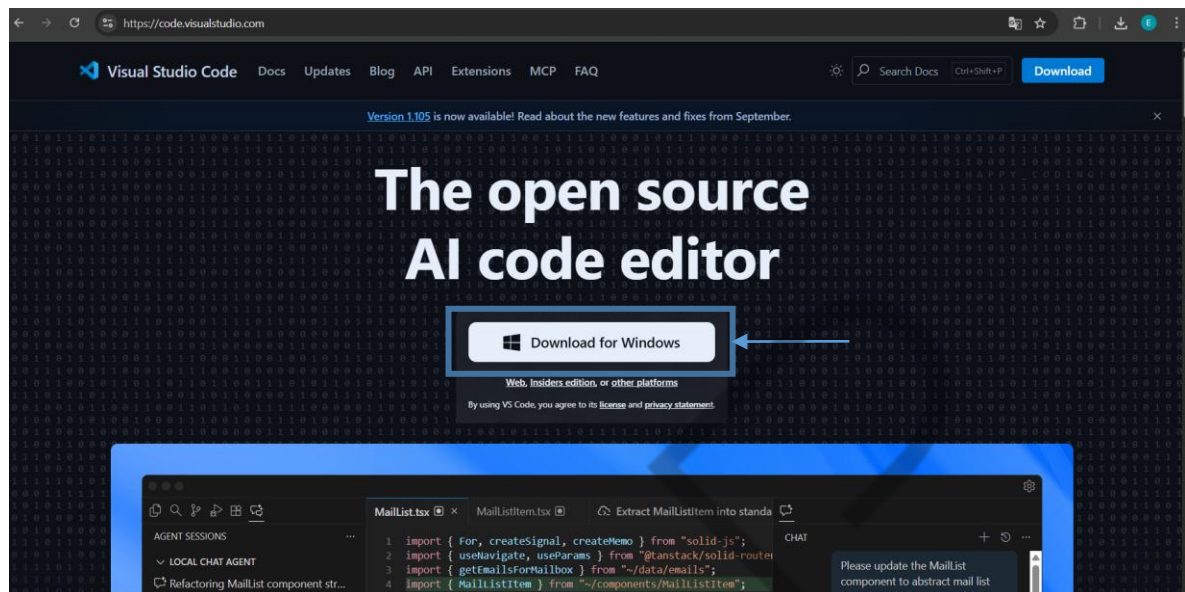


Figura 4 - Página da plataforma VSCode
Fonte: Elaborada pelo autor (2025)

Depois de feito o download, procure o instalador e faça o procedimento de instalação do programa na máquina. Primeiro, deve-se aceitar os termos.

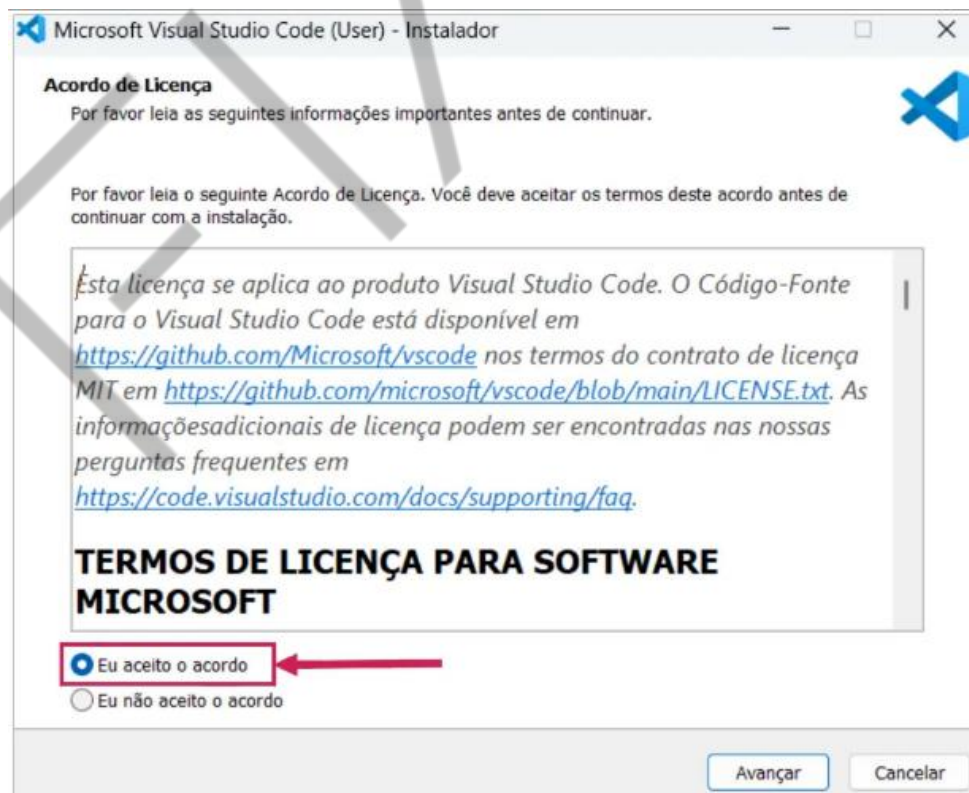


Figura 5 – Termo de instalação do VSCode
Fonte: Elaborada pelo autor (2025)

Agora devemos selecionar as tarefas que a plataforma deve realizar: registrar o código como um editor e sempre adicionar o caminho, mesmo quando sair da plataforma.

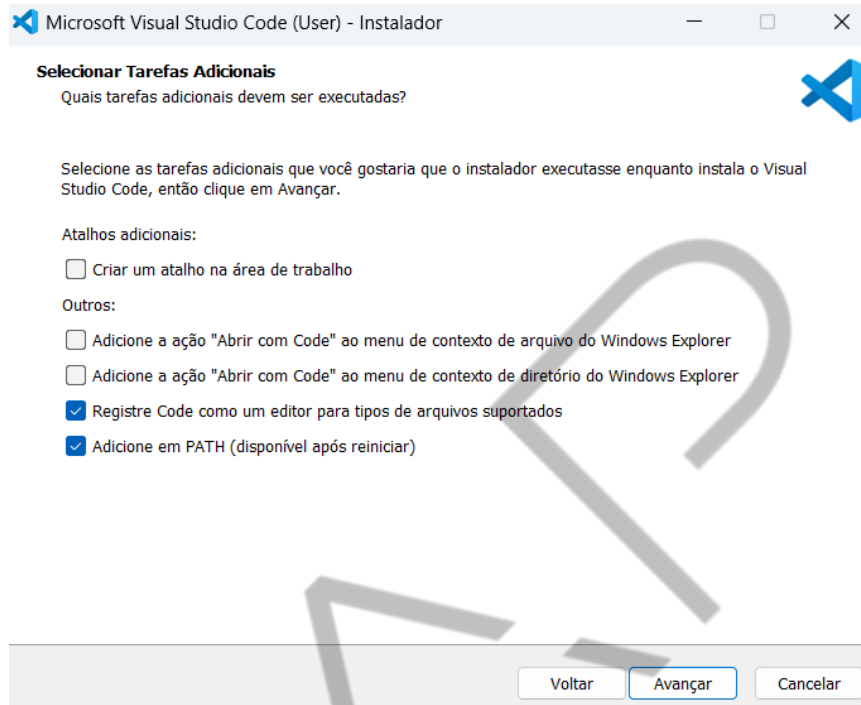


Figura 6 – As tarefas do VSCode
Fonte: Elaborada pelo autor (2025)

A figura a seguir apresenta a confirmação e inicialização da instalação.

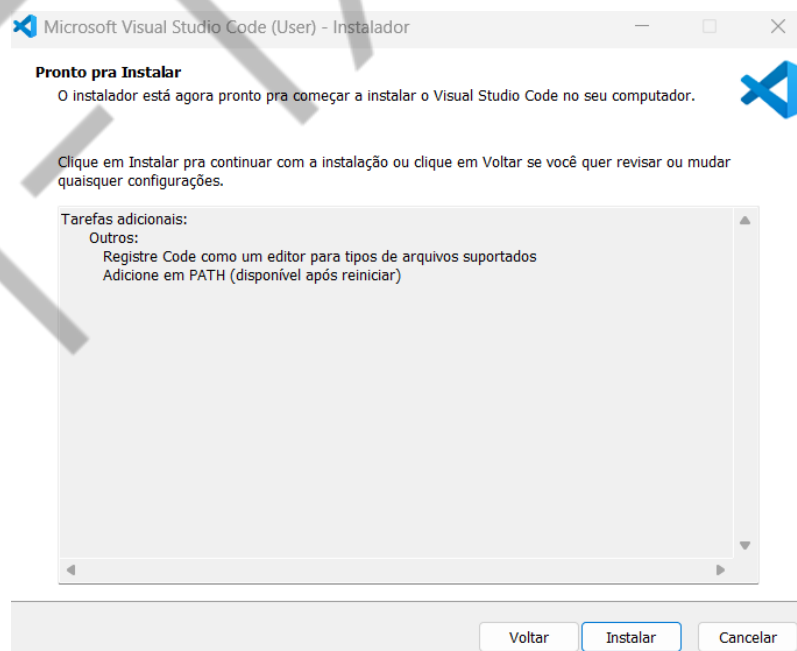


Figura 7 – Preparação para instalar o VSCode
Fonte: Elaborada pelo autor (2025)

Ao completar o processo de instalação, é só clicar em “Concluir”. Vamos colocar a mão na massa!

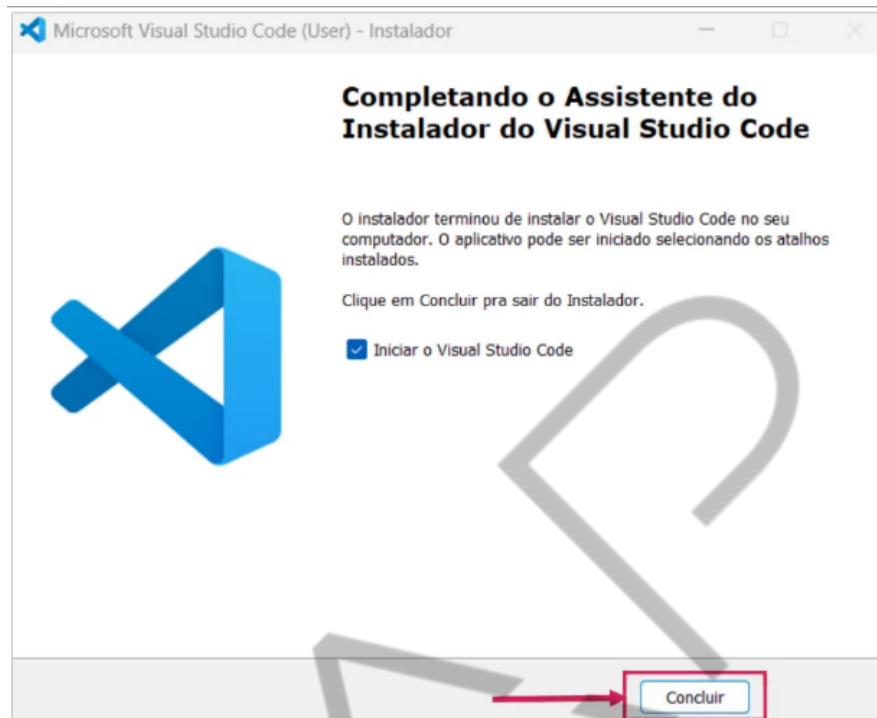


Figura 8 – Conclusão da instalação do VSCode
Fonte: Elaborada pelo autor (2025)

3.3 Instalação por sistema – Linux (Ubuntu/ Debian)

O procedimento é direto no terminal, seguindo os seguintes passos:

```
sudo apt update  
sudo apt install build-essential gdb
```

Código-fonte 6 – Passos para instalação no Linux
Fonte: Elaborado pelo autor (2025)

Os passos acima instalam gcc, g++, make e dependências básicas.

3.4 Instalação por sistema – macOS (clang/LLDB)

Instale as seguintes *command line tools*:

```
xcode-select --install
```

Código-fonte 7 – Passos para instalação no macOS

Fonte: Elaborado pelo autor (2025)

Depois, valide: `clang --version` e `lldb --version`.

3.5 Configurações e extensões no VSCode

Após as instalações, independente do seu sistema operacional, ao abrir a plataforma, teremos a tela principal do VSCode.

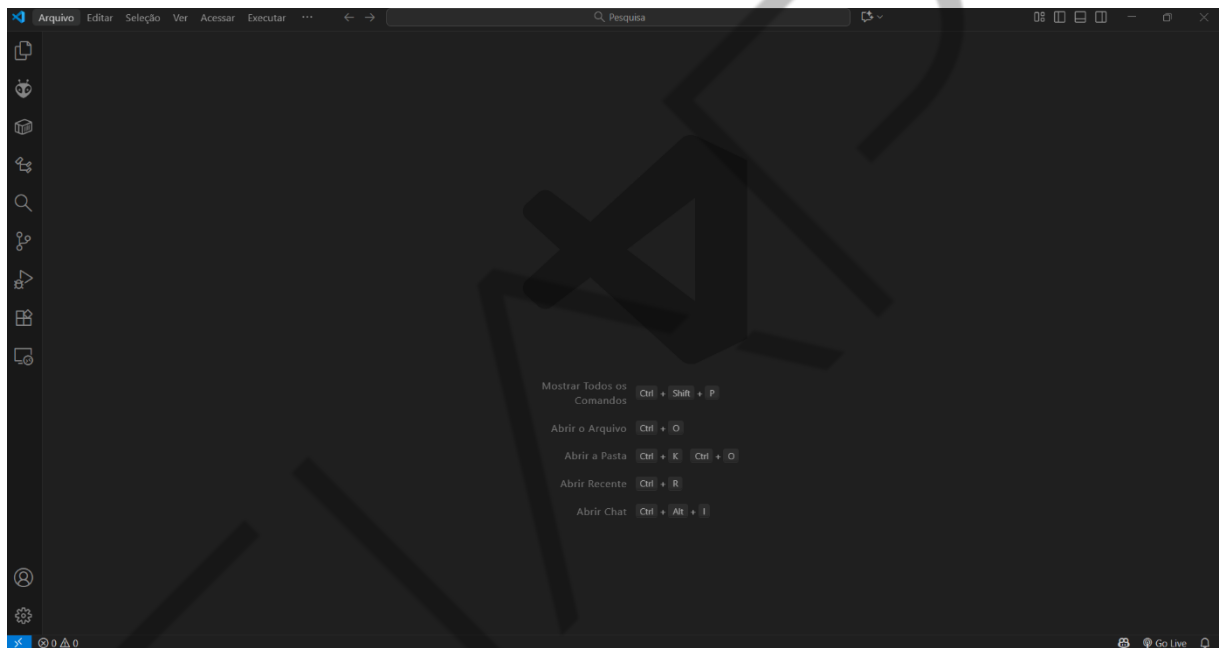


Figura 9 – Tela principal do VSCode

Fonte: Elaborada pelo autor (2025)

Ao clicar no primeiro ícone da esquerda, temos a opção de abrir uma pasta ou clonar um repositório de um projeto do Github. No nosso caso, criaremos uma pasta na área de trabalho e abriremos no VSCode.

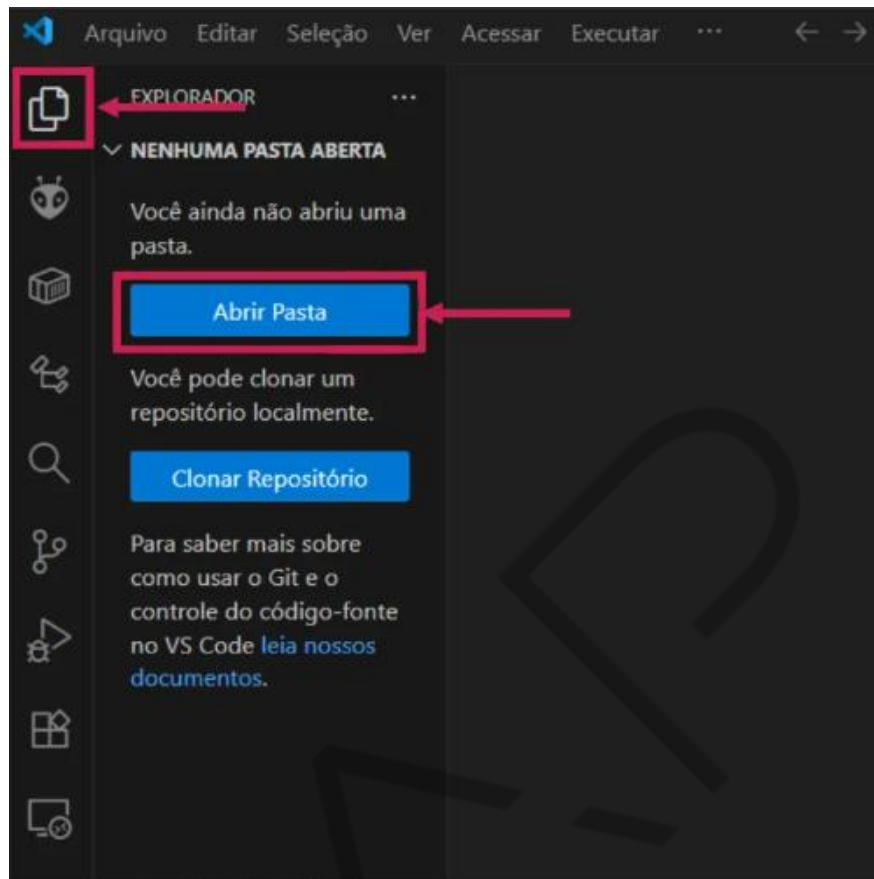


Figura 10 – Seleção de “Abrir Pasta” no VSCode
Fonte: Elaborada pelo autor (2025)

Agora iremos preparar as extensões para compilar os códigos na linguagem C diretamente no VSCode. As extensões selecionadas estão apresentadas na figura a seguir.



Figura 11 – Seleção das pastas do projeto no VSCode
Fonte: Elaborada pelo autor (2025)

O C/C++ da *IntelliSense* é necessário para conseguirmos desenvolver e depurar o código na linguagem apresentando os erros e quaisquer problemas que possam ser encontrados no momento do desenvolvimento. O C/C++ *compile run* é um facilitador no momento de compilar qualquer código de forma rápida e simples.

4 PROGRAMANDO EM C: MEU PRIMEIRO PROGRAMA – HELLO WORLD!

Aqui, daremos o primeiro passo prático na linguagem C: vamos criar um arquivo-fonte, compilar, ligar e executar um programa mínimo que imprime uma mensagem no console. Veremos a estrutura essencial de um programa (a função `main` como ponto de entrada), o papel do cabeçalho `<stdio.h>` para habilitar `printf` e detalhes úteis como o `\n` (quebra de linha) e o código de retorno de `main`, incluindo

o fato de que, nas normas modernas, a ausência de `return 0`; pode ser tratada como sucesso pelo ambiente.

Você também aprenderá a rodar esse exemplo nos principais *toolchains* (GCC/Clang/MinGW/MSVC) e no VS Code com tarefas de *build* e depuração, entendendo cada etapa do ciclo “escrever-compilar-executar”. Ao final, terá dominado o caminho completo do seu primeiro programa em C e poderá repetir o processo para códigos maiores.

Primeiramente, vamos seleccionar a pasta que foi criada na área de trabalho e clicar no ícone de “Novo arquivo”.

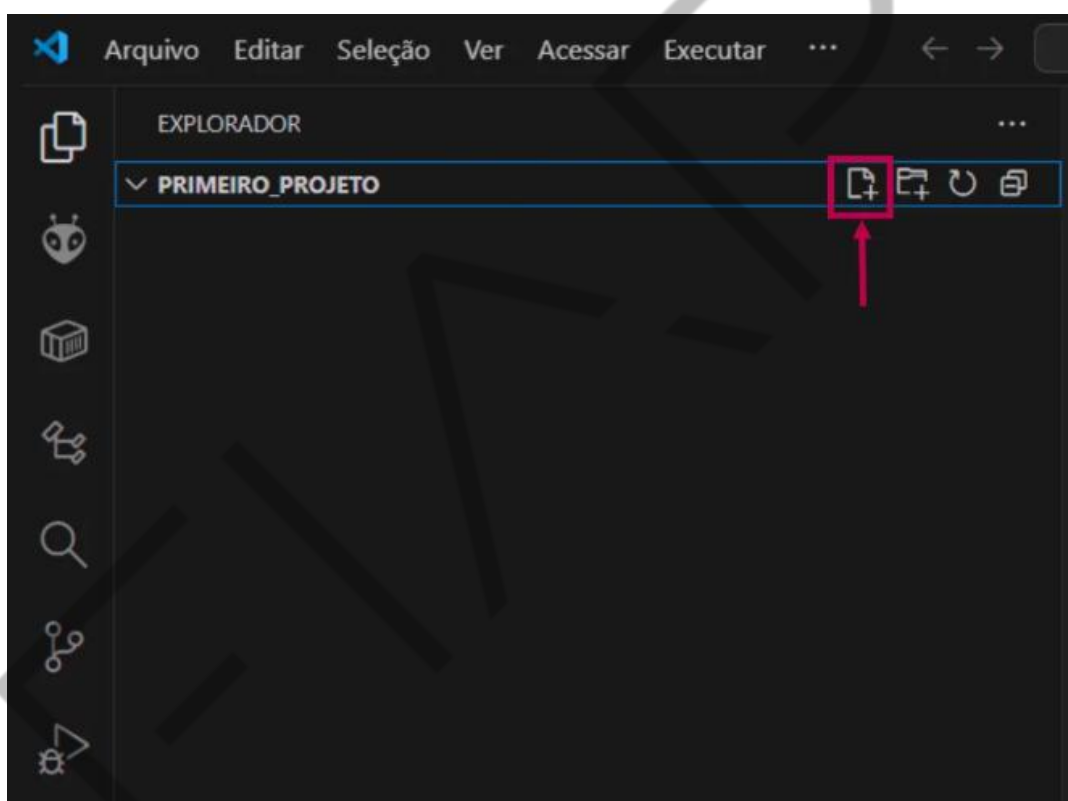


Figura 12 – Criando o primeiro programa no VSCode
Fonte: Elaborada pelo autor (2025)

Ao clicar no ícone indicado, aparecerá um espaço para digitar. Deve-se colocar o nome do arquivo `main.c`. Desse modo, o arquivo abrirá automaticamente para ser editado.

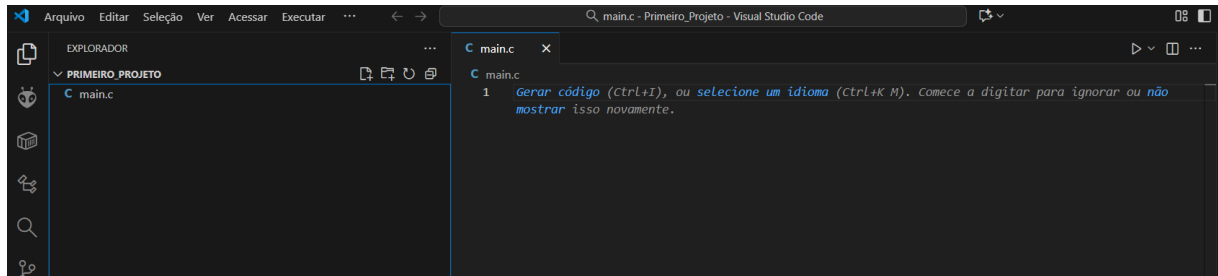


Figura 13 – Criando o primeiro arquivo na pasta do projeto
Fonte: Elaborada pelo autor (2025)

Agora, você deverá digitar as seguintes linhas de código:

```
#include <stdio.h>

int main(void) {
puts("Hello, C from VS Code!"); // ou printf()
return 0;
}
```

Código-fonte 8 – Hello World! Código em C
Fonte: Elaborado pelo autor (2025)

Abaixo está a explicação de cada linha do código:

1. `#include <stdio.h>` - é uma diretiva do pré-processador: antes de compilar, o compilador “cola” no seu código as declarações das funções de E/S (entrada/saída) da biblioteca padrão C, como `puts` e `printf`, que moram no cabeçalho `<stdio.h>`. Sem esse `#include`, o compilador não sabe a assinatura dessas funções;
2. `int main(void) {}` - ponto de entrada do programa: todo programa C começa executando a função `main`. O tipo de retorno deve ser `int` e a forma sem parâmetros é `int main(void)`. O valor retornado por `main` é enviado ao sistema operacional como código de término (a forma alternativa recebe `argc`, `argv` para argumentos de linha de comando);
3. `puts("Hello, C from VS Code!");` - imprime texto seguido de quebra de linha na saída padrão (`stdout`). `puts` escreve a *string* e acrescenta automaticamente `\n` ao final, diferente de `fputs`, que não adiciona a nova

linha. É por isso que não precisamos escrever `\n` aqui (as declarações de `puts` vêm de `<stdio.h>`);

4. `return 0;` - encerra `main` devolvendo 0, que indica sucesso para o ambiente (outros valores $\neq 0$ costumam sinalizar erro). Desde o padrão moderno, se você chegar ao `}` de `main` sem `return`, o efeito é o mesmo de `return 0;` quando o retorno é compatível com `int`. Ainda assim, é uma boa prática manter o `return 0;` explícito para deixar a intenção clara.

A tela que você está programando ficará semelhante a figura a seguir.

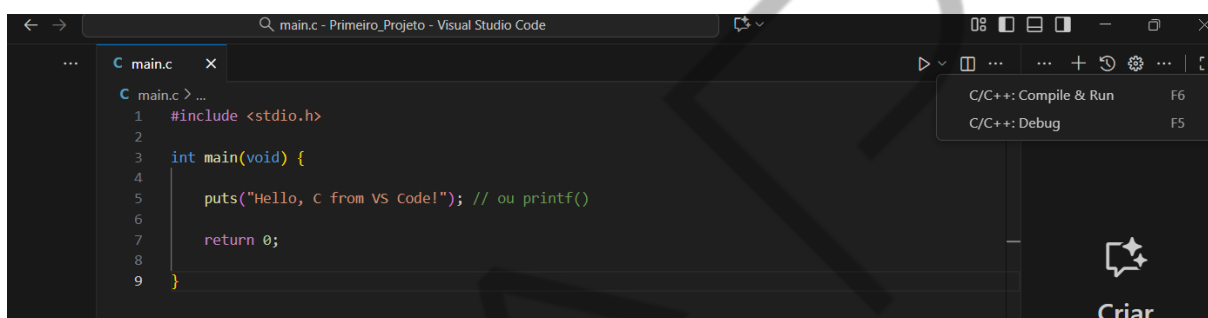


Figura 14 – Tela final do código
Fonte: Elaborada pelo autor (2025)

Para que possamos verificar se o código está correto, devemos clicar no seguinte ícone, em que temos duas opções:

- *C/C++ Debug* (tecla de atalho F5): localizar e corrigir erros (ou bugs) no código-fonte;
- *C/C++ Compile & Run* (tecla de atalho F6): verificar, construir e testar o código-fonte.

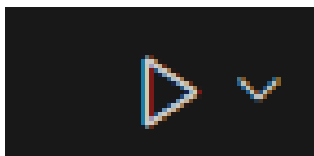
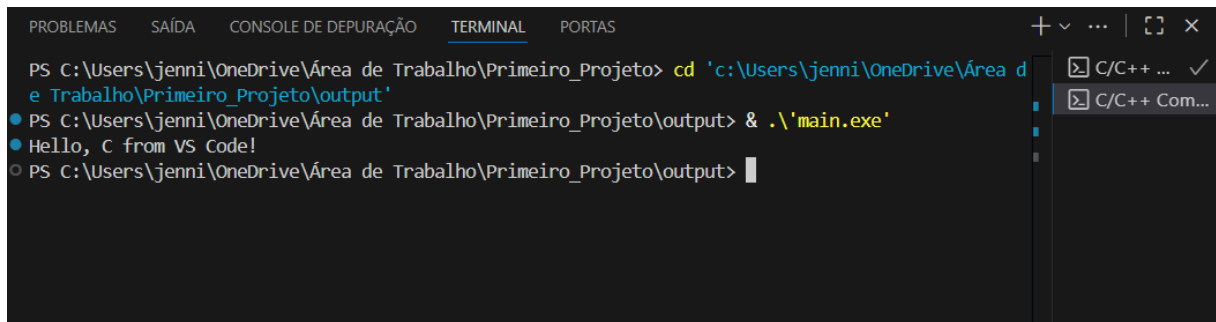


Figura 15 – Ícone para verificação do código
Fonte: Elaborada pelo autor (2025)

Siga a sequência: “**Depurar** → **Compilar** → **Executar**”. Embora o comando *Compile & Run* costume acionar etapas de diagnóstico, é mais seguro realizar cada fase explicitamente quando houver dúvidas.

Após executar *Compile & Run*, aparecerá no seu terminal a tela abaixo.



```
PROBLEMAS  SAÍDA  CONSOLE DE DEPURAÇÃO  TERMINAL  PORTAS
PS C:\Users\jenni\OneDrive\Área de Trabalho\Primeiro_Projeto> cd 'c:\Users\jenni\OneDrive\Área de Trabalho\Primeiro_Projeto\output'
PS C:\Users\jenni\OneDrive\Área de Trabalho\Primeiro_Projeto\output> & .\'main.exe'
Hello, C from VS Code!
PS C:\Users\jenni\OneDrive\Área de Trabalho\Primeiro_Projeto\output> 
```

Figura 16 – Código compilado e executado com sucesso
Fonte: Elaborada pelo autor (2025)

Repare que no terminal aparece o que foi pedido no código. Portanto, ele foi executado com sucesso. Parabéns!

Caso tenha algum problema durante a execução, refaça as etapas anteriores.

5 PROGRAMANDO EM C: LAÇOS DE REPETIÇÃO E CONDICIONAIS

Agora veremos como controlar o fluxo de um programa em C e como modularizar a solução em funções reutilizáveis. Começaremos pelas estruturas condicionais (if, if...else, else if e switch), passando pelos laços de repetição (while, do...while e for) e terminaremos com funções (declaração e parâmetros). Ao final, você conseguirá escolher a estrutura correta para cada problema, escrever código legível e evitar armadilhas comuns (condições com efeito colateral, laços infinitos, passagem incorreta por referência etc.).

Programas reais processam coleções (vetores, arquivos e sensores) e repetem ações até condições de parada (limite de amostras, sentinelas e convergência). Os laços resolvem a repetição controlada e as funções decompõem o problema em unidades menores, reutilizáveis e testáveis.

5.1 Laços de repetição (*loop*): while, do...while e for

Laços de repetição permitem executar um bloco de código várias vezes enquanto uma condição for verdadeira, ou por um número controlado de iterações. Em C, usamos principalmente while (repete 0 ou mais vezes e testa antes), do...while (repete 1 ou mais vezes e testa depois) e for (condensa inicialização, condição e atualização em uma linha, ótimo para contadores).

Escolha o laço conforme a necessidade: quando o número de repetições não é conhecido de antemão, `while`/`do...while` costumam ser mais naturais. Quando há um contador bem definido, `for` é preferível. É importante ter os seguintes cuidados: condição de parada correta, atualização do estado (para não criar laço infinito) e percorrer vetores na convenção $[0, n)$ para evitar *off-by-one*.

5.1.1 `while` — 0 ou mais repetições

Executa enquanto a condição for verdadeira.

```
size_t i = 0;
while (i < n) {
    soma += v[i];
    i++; // ATUALIZAÇÃO: evite laço infinito
}
```

Código-fonte 9 – Exemplo de `while` no código em C
Fonte: Elaborado pelo autor (2025)

- Usos típicos: quando a quantidade de repetições é desconhecida de antemão;
- Erros comuns: esquecer atualização; usar `<=` em vez de `<` e sair da faixa do vetor.

5.1.2 `do...while` — 1 ou mais repetições

Garante ao menos uma execução.

```
int x;
do {
    x = ler_inteiro();
} while (x < 0); // repete até ser não-negativo
```

Código-fonte 10 – Exemplo de `do...while` no código em C
Fonte: Elaborado pelo autor (2025).

5.1.3 `for` — controle compacto

Possui três campos: inicialização, condição e atualização.

```
for (size_t k = 0; k < n; k++) {  
    v[k] *= 2.0f;  
}
```

Código-fonte 11 – Exemplo de (*loop*) for no código em C
Fonte: Elaborado pelo autor (2025)

Boas práticas: usar a convenção `[0, n)` para percorrer vetores (evita *off-by-one*).

5.1.4 break e continue (usar com critério)

O `break` sai do laço e `continue` pula para a próxima iteração.

```
for (size_t k = 0; k < n; k++) {  
    if (v[k] < 0) continue;    // ignora negativos  
    if (v[k] > 1000) break;    // interrompe por limiar  
    acc += v[k];  
}
```

Código-fonte 12 – Exemplo de `break` e `continue`
Fonte: Elaborado pelo autor (2025)

5.2 Condicionais: *true* ou *false*, `if`, `else if` e `else`

Condições permitem que o programa tome decisões com base em testes lógicos. Em C, zero é falso e qualquer valor diferente de zero é verdadeiro. A partir disso, usamos `if`, `else if` e `else` para executar caminhos alternativos de código, e `switch` para seleção múltipla quando a escolha depende de um único valor integral (como `char`, `int` ou `enum`). Operadores relacionais (`==`, `!=`, `<`, `>`, `<=`, `>=`...) e lógicos (`&&`, `||` e `!`) compõem as condições, com curto-circuito em `&&/||` para evitar avaliações desnecessárias (e perigosas). Aqui, você aprenderá a escrever condições claras, evitar armadilhas comuns (como `=` no lugar de `==` e `switch` sem `break`) e a escolher entre `if/else` e `switch` com base em clareza, manutenção e desempenho.

5.2.1 Verdadeiro e falso em C

Em C, zero é falso e qualquer valor diferente de zero é verdadeiro. Além disso, expressões relacionais (<, >=, ==...) e lógicas (&&, || e !) produzem int com valor 0 ou 1.

5.2.2 if, else if, else

Use para seleção binária ou em cascata.

```
int sinal(int n) {  
    if (n < 0) { // bloco executa se a expressão for != 0  
        return -1;  
    } else if (n == 0) {  
        return 0;  
    } else {  
        return 1;  
    }  
}
```

Código-fonte 13 – Exemplo de if, else if e else no código em C
Fonte: Elaborado pelo autor (2025).

Boas práticas:

- Sempre use chaves {} mesmo para um único comando: isso evita o erro do “*dangling else*” e facilita a manutenção;
- Prefira *guard clauses* (retornos antecipados) para reduzir aninhamento;
- Cuidado com atribuição = no lugar de igualdade == em condições.

5.2.3 switch case (seleção múltipla)

É interessante utilizar quando a decisão depende de um único valor integral (char, int e enum).

```
char conceito_por_nota(int n) {  
    switch (n)  
    { case 10: case 9: return 'A';
```

```
    case 8: case 7: return 'B';  
    case 6: return 'C';  
    case 5: case 4: return 'D';  
    default: return 'E';  
}  
}
```

Código-fonte 14 – Exemplo de switch case no código em C

Fonte: Elaborado pelo autor (2025)

Regras e cuidados:

- Cada case precisa de um valor constante (em C não há intervalos nativos, como case 1..3);
- Use break quando o fluxo não terminar no return; pois fallthrough acidental é um erro comum;
- Inclua default para cobrir casos imprevistos (útil para enum).

5.3 Funções: Protótipo, definição, chamada e tipos

Funções são blocos nomeados de código que recebem parâmetros, executam uma tarefa e retornam (ou não) um valor. Elas tornam o programa modular, reutilizável e testável, permitindo dividir problemas grandes em partes menores e claras. Em C, toda função tem um tipo de retorno, um nome, parâmetros tipados entre parênteses e um corpo entre chaves. Além disso, a função main é o ponto de entrada do programa.

A linguagem usa passagem por valor (cópia dos argumentos). Portanto, para “devolver” resultados adicionais ou modificar dados do chamador, empregamos ponteiros. Boas práticas incluem declarar protótipos em arquivos .h, implementar em .c, usar nomes descritivos e manter uma responsabilidade por função.

5.3.1 Protótipo, definição e chamada

- Protótipo declara a assinatura para o compilador (normalmente no .h);
- Definição fica no .c;

- Chamada usa a função com argumentos do tipo correto.

```
/* mathx.h (Protótipo)*/
#ifndef MATHX_H
#define MATHX_H
int soma(int a, int b);
#endif

/* mathx.c (Definição)*/
#include "mathx.h"
int soma(int a, int b) { return a + b; }

/* main.c (Chamada)*/
#include "mathx.h"
int main(void) { return soma(2,3); }
```

Código-fonte 15 – Exemplo de protótipo, definição e chamada no código em C
Fonte: Elaborado pelo autor (2025)

5.3.2 Função com retorno e com parâmetros

A ideia dessa função é receber duas variáveis numéricas do tipo inteiro e retornar a soma dessas variáveis de forma inteira também.

```
#include <stdio.h>

int soma(int a, int b) { // recebe dois int e devolve um int
    return a + b;
}

int main(void) {
    int r = soma(2, 3);
    printf("soma(2,3) = %d\n", r); // 5
    return 0;
}
```

Código-fonte 16 – Exemplo de função com retorno no código em C
Fonte: Elaborado pelo autor (2025)

5.3.3 Função com retorno e sem parâmetros

Essa função é aplicada, normalmente, para alguma requisição de texto ou número informativos para o usuário. No exemplo abaixo, o main chama a função `resposta_padrao()`, que exibe o valor 42 sem parâmetros.

```
#include <stdio.h>

int resposta_padrao(void) { // não recebe nada, devolve um int
    return 42;
}

int main(void) {
    printf("resposta = %d\n", resposta_padrao()); // 42
    return 0;
}
```

Código-fonte 17 – Exemplo de função somente com retorno no código em C
Fonte: Elaborado pelo autor (2025)

5.3.4 Função sem retorno (void) e com parâmetros

Essa função realiza uma ação e não devolve valor. O código abaixo apresenta um exemplo na qual a função recebe uma mensagem e replica duas vezes, porém não retorna nada, pois não existe a requisição `return` para devolver algo e ser utilizado em outro momento.

```
#include <stdio.h>

void imprime_duas_vezes(const char *msg) { // só executa um
    efeito (imprimir)
    printf("%s\n", msg);
    printf("%s\n", msg); }

int main(void) {
    imprime_duas_vezes("Olá!");
}
```

```
return 0;
}
```

Código-fonte 18 – Exemplo de função somente com parâmetros no código em C
Fonte: Elaborado pelo autor (2025)

5.3.5 Função sem retorno (void) e sem parâmetros

Esse tipo de função é comumente utilizado para introduzir uma interface de usuário, como um cabeçalho.

```
#include <stdio.h>
void cabecalho(void) { // não recebe nada e não retorna nada
    puts("=== Meu Programa ===");
}

int main(void) {
    cabecalho();
    return 0;
}
```

Código-fonte 19 – Exemplo de função sem parâmetros e sem retorno no código em C
Fonte: Elaborado pelo autor (2025)

Agora que dominamos condicionais, laços e funções, ou seja, já sabemos decidir, repetir e organizar o código, estamos prontos para tratar como os dados são estruturados na memória. A seguir, veremos como representar coleções de elementos e operar sobre elas com eficiência, começando por pilhas (LIFO) e filas (FIFO), até chegar às listas encadeadas. Você perceberá como esses conceitos são a base para implementar *push/pop*, *enqueue/dequeue* e *inserções/remoções* com segurança e clareza.

6 PROGRAMANDO EM C: ESTRUTURAS LINEARES – LISTAS, FILAS E PILHAS

As estruturas lineares são jeitos de organizar dados em sequência, um depois do outro, para guardar, procurar e retirar informações. Veremos os seguintes tipos: pilha (*stack*), fila (*queue*) e lista encadeada (*linked list*).

Porém, para entendermos essas estruturas, primeiros precisamos analisar os vetores.

6.1 O que é um vetor?

Pense em um gaveteiro numerado. Cada gaveta tem um número (0, 1, 2, ...) e você pega direto a gaveta que quiser.

Na linguagem C: um array é uma sequência contígua na memória e o tamanho é fixo enquanto ele “vive”. O acesso é feito por meio de colchetes, como `v[2]`.

```
int v[4] = {10, 20, 30, 40};  
printf("%d\n", v[2]); // imprime 30
```

Código-fonte 20 – Exemplo de vetor no código em C
Fonte: Elaborado pelo autor (2025)

É importante pensar da seguinte maneira: esse conceito é ótimo para pegar um item pela posição. Se precisar “abrir espaço” no meio, pode ser preciso mover outros itens (como arrastar gavetas).

6.2 Pilha (*stack*)

Aqui temos a mesma ideia de uma pilha de pratos: você mexe só no topo. Portanto, o último que entrou é o primeiro que sai; essa operação chama-se LIFO (*Last In, First Out*). As operações básicas relacionadas são: empilhar (*push*), desempilhar (*pop*) e espiar o topo (*top/peek*).

Em C, a linha de código é vista da seguinte maneira:

```
int s[100], top = 0;      // top = quantidade  
void push(int x){ s[top++] = x; }  
int pop(void){ return s[--top]; }
```

Código-fonte 21 – Exemplo de pilha (*stack*) no código em C
Fonte: Elaborado pelo autor (2025)

Nessa estrutura deve-se pensar da seguinte maneira: é perfeita para “desfazer” ações (voltar uma etapa), fazer a análise de expressões, histórico de telas etc.

6.3 Fila (Queue)

Essa estrutura pode-se associar a fila de mercado: quem chega primeiro, sai primeiro. Essa operação se chama FIFO (*First In, First Out*). Além disso, as operações básicas relacionadas são: entrar (*enqueue*), sair (*dequeue*) e ver o primeiro (*front*).

```
#define CAP 4
int q[CAP], head=0, tail=0, size=0;

void enqueue(int x){ q[tail]=x; tail=(tail+1)%CAP; size++;
}

int dequeue(void){ int x=q[head]; head=(head+1)%CAP; size--; return x; }
```

Código-fonte 22 – Exemplo *buffer* “circular” da estrutura fila (queue) no código em C
Fonte: Elaborado pelo autor (2025)

Como o exemplo acima, essa estrutura é ótima para atendimentos e filas de tarefas, já que o “circular” evita ter que mover tudo quando alguém sai da parte da frente.

6.4 Lista encadeada (*linked list*)

A lista encadeada é como se fosse uma trilha de placas com setas: cada placa guarda um valor e uma seta para a próxima. Dentro de C, cada “placa” é um nó com dados + ponteiro next. Os nós não precisam estar grudados na memória, já que é possível ligar um ao outro com o ponteiro.

```
#include <stdlib.h>
typedef struct Node {
    int x;
    struct Node *next;
} Node;
```

```
Node* push_front(Node *head, int x){
    Node *n = malloc(sizeof *n);
    if(!n) return head; // checagem mínima
    n->x = x; n->next = head;
    return n; // novo primeiro nó
}
```

Código-fonte 23 – Exemplo de lista encadeada (linked list) no código em C
Fonte: Elaborado pelo autor (2025)

Essa estrutura é ideal quando há inserções ou remoções frequentes (é como “trocar setas”), mas, para chegar ao 50º elemento, é preciso seguir as setas uma a uma.

Para memorizar melhor, guarde as seguintes imagens mentais: gaveteiro (vetor), pratos (pilha), fila do mercado (queue) e placas com setas (lista encadeada). Com essas quatro referências, você já resolve muitos problemas do dia a dia em programação C.

CONCLUSÃO

Ao longo deste capítulo, você construiu uma base sólida em *data structures and algorithms* a partir da linguagem C, compreendendo desde seus fundamentos históricos e conceituais até a aplicação prática dos principais recursos da linguagem. Foram explorados aspectos essenciais como tipos de dados, variáveis, operadores, expressões, controle de fluxo com condicionais e laços de repetição, além da organização do código por meio de funções. Esse percurso permitiu entender não apenas como programar em C, mas por que determinadas decisões de projeto são importantes para escrever códigos corretos, legíveis e portáveis, alinhados às boas práticas da computação moderna.

Ao fim, avançamos para o estudo das estruturas lineares, vetores, pilhas, filas e listas encadeadas, conectando conceitos abstratos a exemplos práticos e analogias do cotidiano. Essas estruturas representam o primeiro grande passo para organizar dados de forma eficiente na memória e resolver problemas reais de maneira

estruturada. Com esse conhecimento, você está preparado para compreender operações fundamentais como inserção, remoção e acesso a dados, além de enxergar como esses mecanismos servem de base para algoritmos e estruturas mais avançadas.



REFERÊNCIAS

CPPREFERENCE. **Arithmetic types.** Disponível em: <https://en.cppreference.com/w/c/language/arithmetic_types.html>. Acesso em: 05 jan. 2025.

HARBISON, S. P. & STEELE, G. L., Jr. (2002). **C: A Reference Manual** (5th ed.). Prentice Hall.

ISO/IEC 9899. **C: N1570, working drafts e norma 2023.** Disponível em: <https://web.cs.dal.ca/~vlado/pl/C_Standard_2011-n1570.pdf>. Acesso em: 05 jan. 2025.

ISO/IEC. **ISO/IEC 9899:201x — Programming Languages — C** (Committee Draft N1570, C11). Disponível em: <https://web.cs.dal.ca/~vlado/pl/C_Standard_2011-n1570.pdf>. Acesso em: 05 jan. 2025.

ISO/IEC JTC1/SC22/WG14. **Programming Languages — C** (ISO/IEC 9899). Página oficial do WG14 (histórico C90, C99, C11, C17, C23) e drafts públicos. Disponível em: <<https://www.open-std.org/jtc1/sc22/wg14/>>. Acesso em 05 jan. 2025.

KERNIGHAN, B. W. & RITCHIE, D. M. (1988). **The C Programming Language** (2nd ed.). Prentice-Hall.

MATZ, M.; HUBICKA, J.; JAEGER, A.; MITCHELL M. (2012). **AMD64 SysV psABI (LP64):** long/ponteiros 64 e long double ocupa 16 bytes (80-bit significativos). Disponível em: <https://refspecs.linuxbase.org/elf/x86_64-abi-0.99.pdf>. Acesso em: 05 jan. 2025.

RITCHIE, D. M. (1993). *The Development of the C Language* (HOPL-II). Disponível em: <<https://www.nokia.com/bell-labs/about/dennis-m-ritchie/chist.pdf>>. Acesso em: 05 jan. 2025.

RITCHIE, D. M. & THOMPSON, K. (1974). **The UNIX time-sharing system.** Communications of the ACM, 17(7), 365–375. Disponível em: <<https://dl.acm.org/doi/10.1145/361011.361061>>. Acesso em: 05 jan. 2025.